

Simulating Randomness

CS 313 – Computational Science
1st Sem., S.Y. 2022 - 2023



Computer Simulation

- Having computer program imitate reality, in order to study situations and make decisions
- Applications?



Use simulations if

- Not feasible to do actual experiments
- System does not exist
- Cost of actual experiments prohibitive
 - Money
 - Time
 - Danger
- Want to test alternatives



Disadvantages of Computer Simulations

- The simulation may be expensive in time or money to develop.
- Because it is impossible to test every alternative, we can provide good solutions but not the best solution.
- The results may be difficult to verify because often we do not have real-world data.
- We cannot be sure we understand what the simulation actually does.
- When a simulation is probabilistic, involving an element of chance, we should be careful of our conclusions.



Element of Chance

- At the core of most simulations is *random number generation*.
- The computer generates a sequence of numbers, called random numbers, or *pseudorandom* numbers.
- An algorithm actually produces the numbers, so they are not really random, but they *appear to be random*.
- Because of the element of chance, we often call a simulation a Monte Carlo simulation.



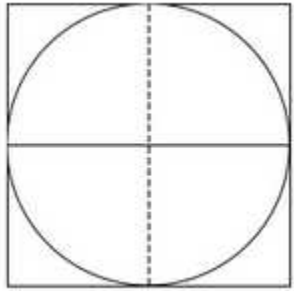
Monte Carlo simulation

- Probabilistic model involving element of chance
- Not deterministic
- Uses random number generator



Estimating Pi by Throwing Darts

Monte Carlo methods employ statistical algorithms to estimate an unknown quantity. For us, this unknown quantity is π . The method starts by inscribing a circle inside of a square.



Think of the circle as a dartboard mounted on a square block of wood. Close your eyes and throw darts towards the board. Assume that all your darts stay within the surrounding square. Count the number of darts that land on the dart board. What percentage of darts hit the board?

Randomly generate points in the square and count the number of these points that fall within the inscribed circle. When you are done generating the points, compute the ratio of points in the circle to the total number of points.

```
In [1]: import turtle
import random

r = 200 #radius of the circle|
t1 = turtle.Turtle()
t1.speed(0)
t1.hideturtle()

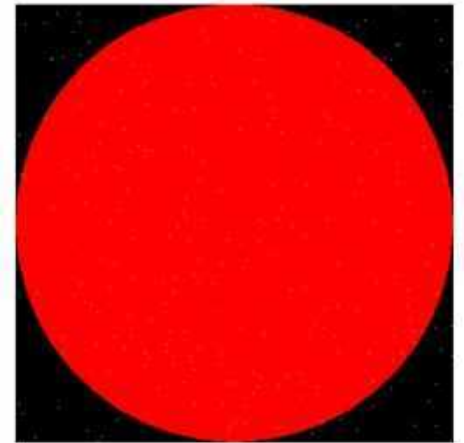
# Draw the square centered at (0,0), side length == r
t1.pu()
t1.goto(-200, -200)
t1.pd()
t1.begin_fill()
for _ in range(4):
    t1.fd(400)
    t1.lt(90)
t1.pu()
t1.end_fill()

# Draw the inscribed circle
# Radius = 200, center = (0,0)

t1.goto(0, -r)
t1.color('red')
t1.begin_fill()
t1.pd()
t1.circle(r)
t1.end_fill()

# Randomly pick points inside the square
# Count the number of points inside the circle: distance from (0, 0) < 200

num_points = 1000
num_in_circle = 0
t1.color('yellow')
t1.pu()
for _ in range(num_points):
    x = random.randint(-200, 200)
    y = random.randint(-200, 200)
    t1.goto(x, y)
    t1.dot(2)
    if(x*x + y*y)**0.5 < r: # test if point is inside the circle
        num_in_circle += 1
t1.goto(-145, -250)
t1.color('black')
t1.write("Ratio of points in circle to points in square = " + \
        str(num_in_circle/num_points), font=("Arial", 14, "normal"))
turtle.done()
```



Multiplicative Linear Congruential Method

- $random_0 = 10$
- $random_n = (7 * random_{n-1}) \bmod 11, \text{ for } n > 0$
- Generates?
 - 10, 4, 6, 9, 8, 1, 7, 5, 2, 3



Multiplicative Linear Congruential Method

- $random_0 = seed$
- $random_n = (multiplier * random_{n-1}) \bmod modulus$, for $n > 0$
- $modulus$ often largest integer comp. can store, such as

$$2^{31} - 1 = 2,147,483,647$$

- One multiplier: 16,807



Random floating point number

- For random number, $rand$, with $0.0 \leq r$ and < 1.0
 - $rand = random_n / modulus$
- Example $r_0 = 10$, $r_n = (7 * r_{n-1}) \bmod 11$, for $n > 0$
 - Integers: 10, 4, 6, 9, 8, 1, 7, 5, 2, 3
 - Floating point: 10/11, 4/11, ..., 3/11



Random floating point number

$$\min \leq r < \max$$

- For $0 \leq rand < 1$
 - $r = (\max - \min) rand + \min$
- Example: random floating point number between 20.0 and 26.3?
 - $6.3rand + 20.0$



Random integer $\min \leq n \leq \max$

- For $0 \leq \text{rand} < 1$
 - $n = \text{int}((\max - \min + 1) \text{rand} + \min)$
- Example: random integer between 20 and 26, inclusively?
 - $\text{int}(7 * \text{rand} + 20)$



Random Numbers from Various Distributions

CS 313 – Computational Science
1st Sem., S.Y. 2022 - 2023



Distribution of numbers

- Description of portion of times each possible outcome or each possible range of outcomes occurs on the average over many trials



Uniform distribution

- Just as likely to return value in any interval
- In list of many random numbers, on the average each interval contains same number of generated values



Discrete Distribution

- Distribution with discrete values
- **Probability function (or density function or probability density function)**
 - Returns probability of occurrence of particular argument value



Discrete Distribution

- To Generate Random Numbers in Discrete Distribution with Equal Probabilities for Each of n Events
 - Generate uniform random integer from a sequence of n integers, where each integer corresponds to an event



Discrete Distribution

To generate random numbers in discrete distribution with probabilities $p_1, p_2, \dots, p_n$ for events $e_1, e_2, \dots, e_n$, respectively, where $p_1 + p_2 + \dots + p_n = 1$

Generate *rand*, uniform random floating point number in $[0, 1)$

If $rand < p_1$, then return e_1

else if $rand < p_1 + p_2$, then return e_2

...

else if $rand < p_1 + p_2 + \dots + p_{n-1}$, then return e_{n-1}

else return e_n



Continuous Distribution

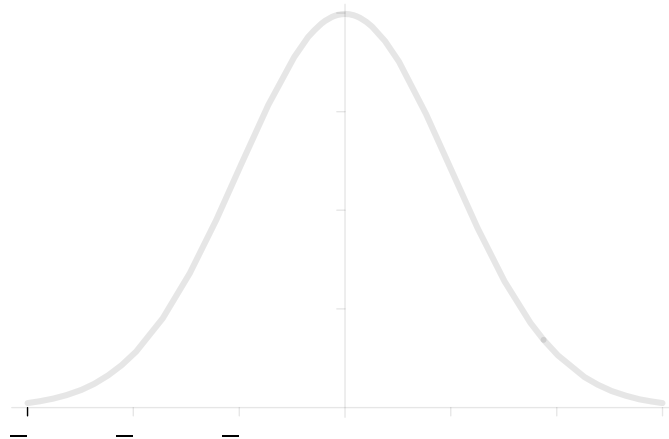
- Distribution with continuous values
- **Probability function (or density function or probability density function)**
 - Indicates probability that given outcome falls inside specific range of values



Normal or Gaussian distribution

- Probability density function, where μ is mean and σ is standard deviation:

$$\frac{1}{\sqrt{2\pi}\sigma} e^{-\frac{(x-\mu)^2}{2\sigma^2}}$$



Box-Muller-Gauss Method

- For normal distribution with mean μ and standard deviation σ
 - Compute $b \sin(a) + \mu$ and $b \cos(a) + \mu$ where
 - $a =$ uniform random number in $[0, 2\pi)$
 - $rand =$ uniform random number in $[0, 1)$
 - $b = \sigma \sqrt{-\ln(rand)}$

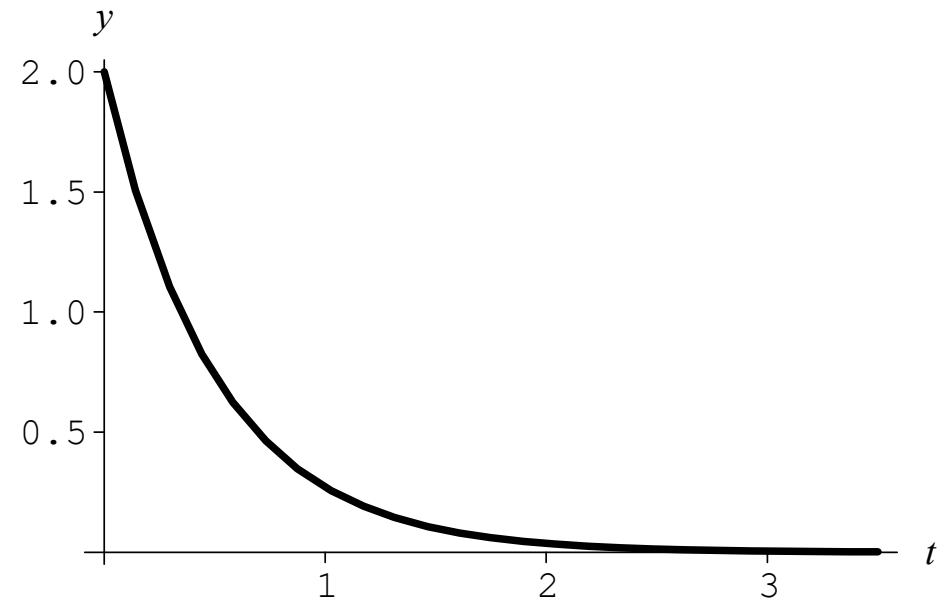


Exponential Distributions

- Probability density functions:

- $f(t) = |r| e^{rt}$ with $r < 0$ and $t > 0$ or $f(t) = |r| e^{rt}$ with $r > 0$ and $t < 0$

- Graph of $f(t) = 2e^{-2t}$



Exponential Method

- For probability density function $|r|e^{rt}$ with $r < 0$ and $t > 0$ or $f(t) = |r|e^{rt}$ with $r > 0$ and $t < 0$
 - Compute $\ln(rand)/r$,
 - where $rand$ = a uniform random number in $[0.0, 1.0)$



Rejection Method - When other methods do not apply

- For random numbers in interval $[a, b)$ for distribution $f(x)$
 - If $f(\text{randInterval}) > \text{randUpperBound}$, then return randInterval , where
 - randInterval - uniform random number in interval $[a, b)$
 - randUpperBound - uniform random number in $[0, \text{upper bound for } f)$
 - else repeat the process



Reference

- A.B.Shiflet, G.W.Shiflet, Introduction to Computational Science, 2nd Edition, 2014, Princeton University Press.

